

Основы глубинного обучения

Лекция 3

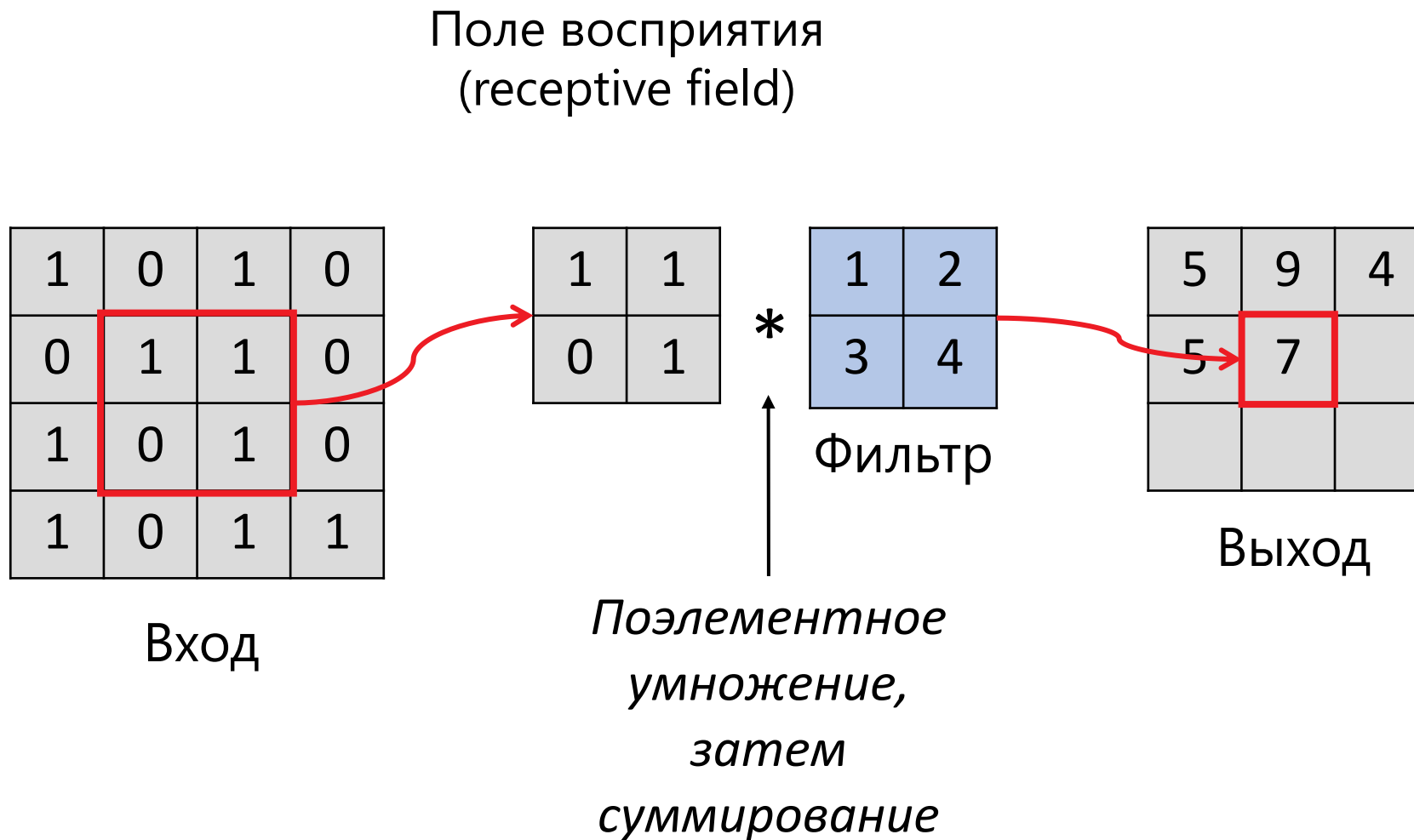
Свёрточные сети

Евгений Соколов

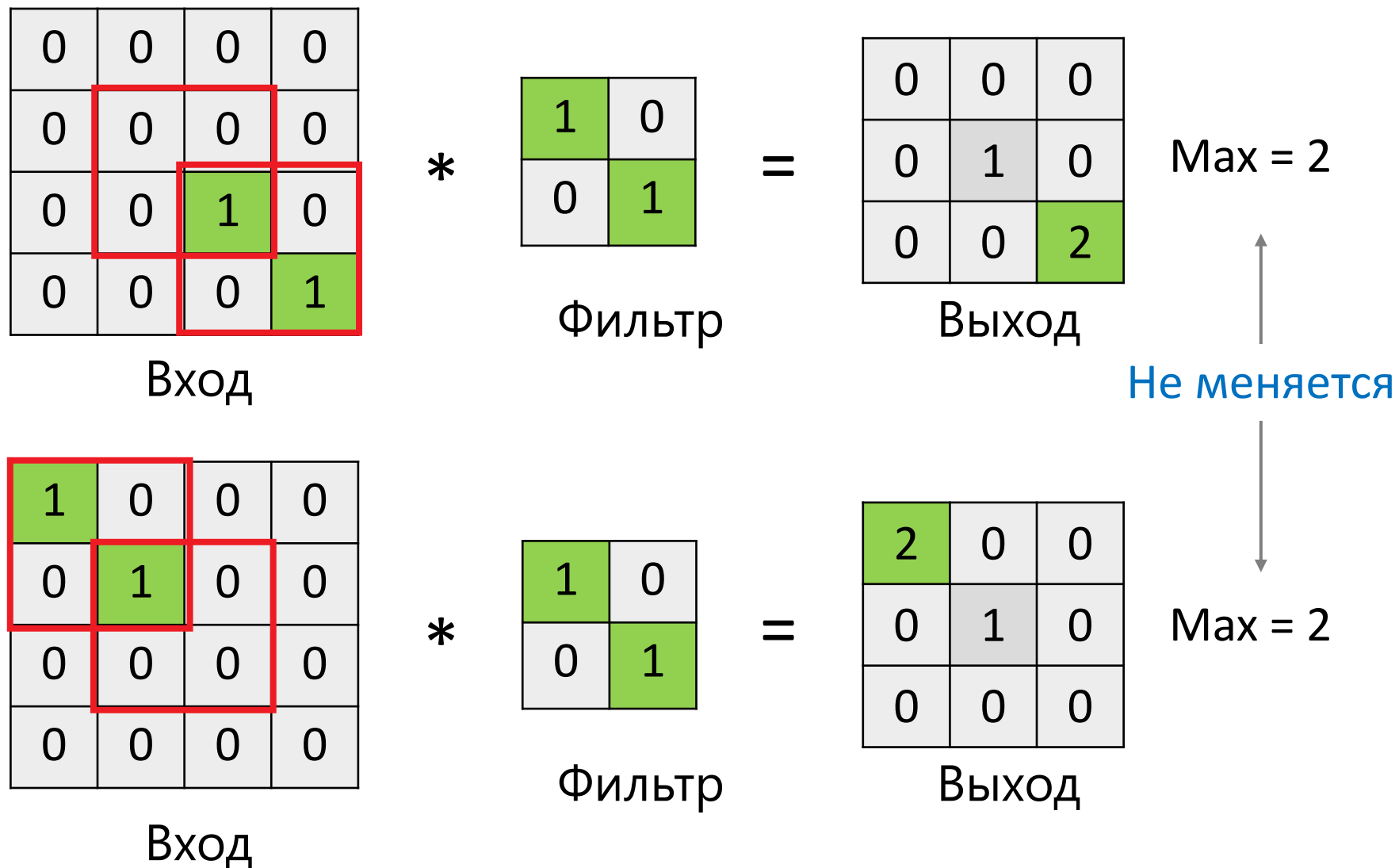
esokolov@hse.ru

НИУ ВШЭ, 2026

Свёртка



Максимум свёртки инвариантен к сдвигам

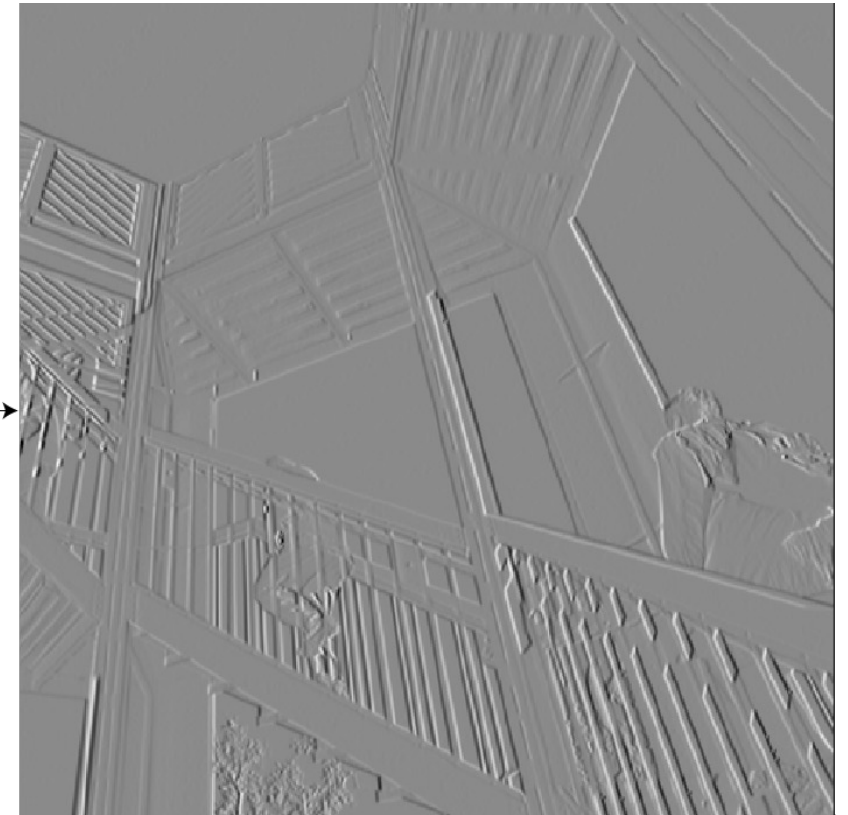


Свёртки в компьютерном зрении



$$\begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix}$$

Horizontal Sobel kernel



Свёртка

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \left(K(i, j) \text{Im}^{in}(x + i, y + j) \right) + b$$

Свёртка

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \left(K(i, j) \text{Im}^{in}(x + i, y + j) \right) + b$$

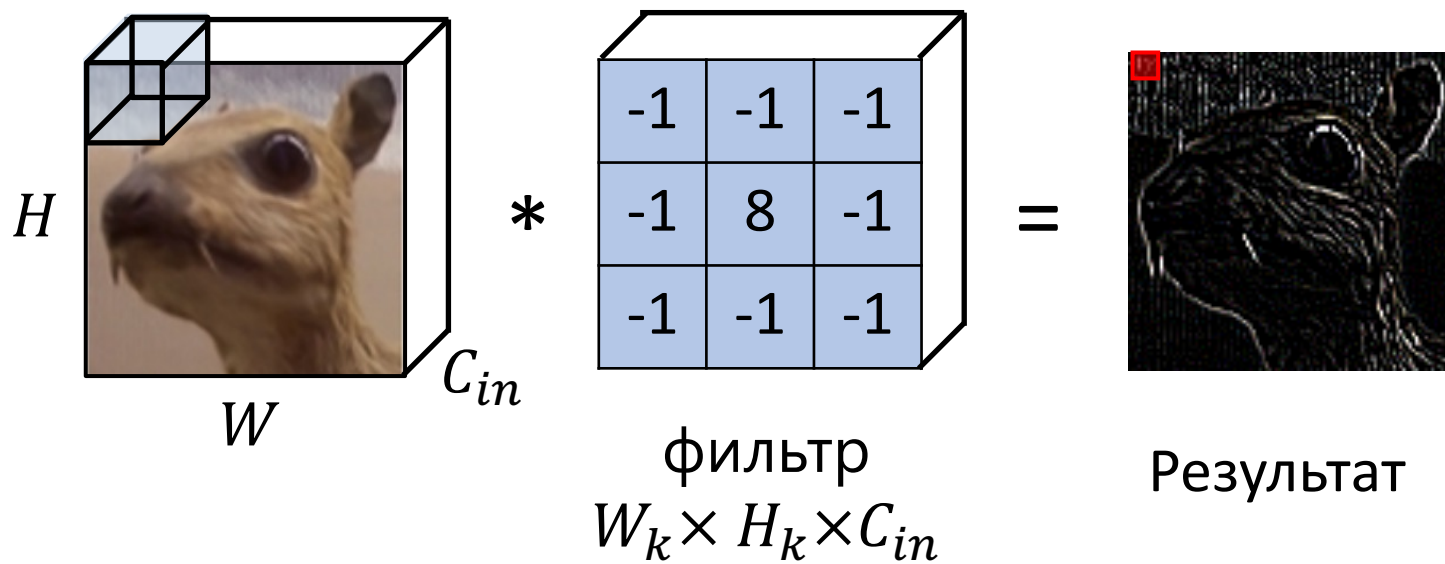
- Пиксель в результирующем изображении зависит только от небольшого участка исходного изображения (local connectivity)
- Веса одни и те же для всех пикселей результирующего изображения (shared weights)

Свёртка

- Обычно исходное изображение цветное!
- Это означает, что в нём несколько каналов (R, G, B)
- Учтём в формуле:

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b$$

Свёртка



Свёртка

- Одна свёртка выделяет конкретный паттерн на изображении
- Нам интересно искать много паттернов
- Сделаем результат трёхмерным:

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b_t$$

Число параметров

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b_t$$

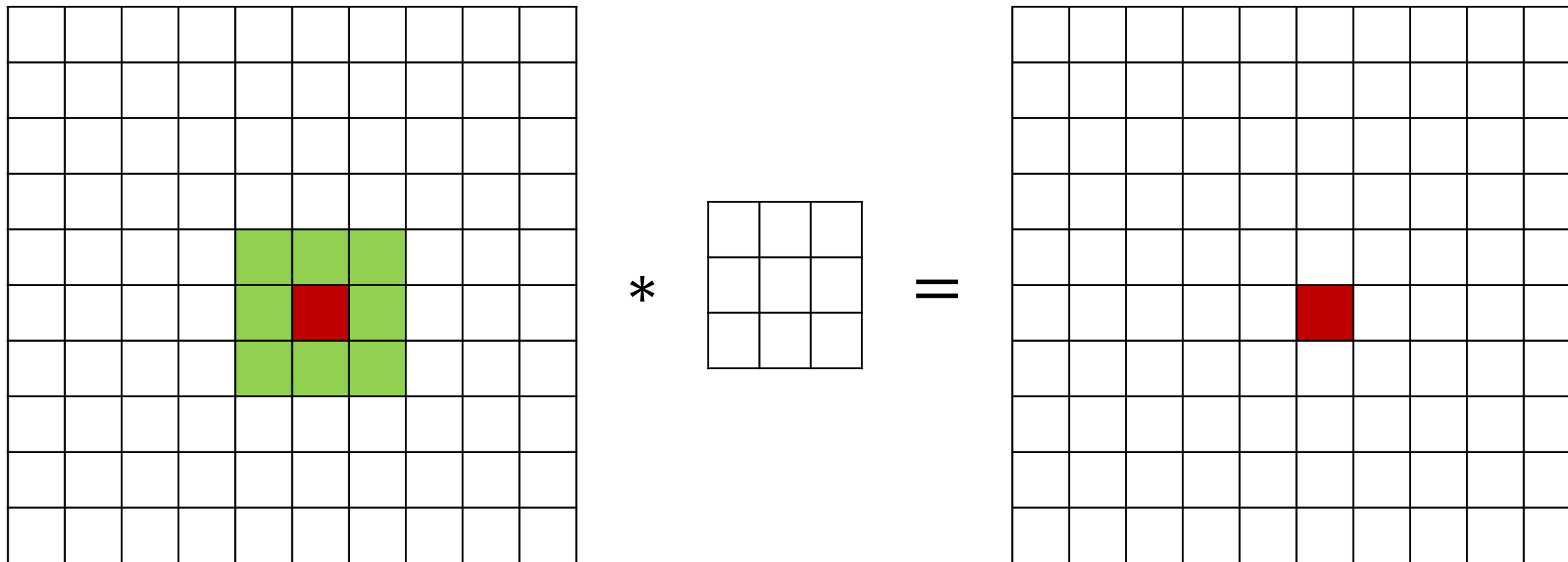
- Обучается только фильтр
- $((2d + 1)^2 * C + 1) * T$ параметров
- Как из этого сделать модель — обсудим позже

Receptive field

Receptive field

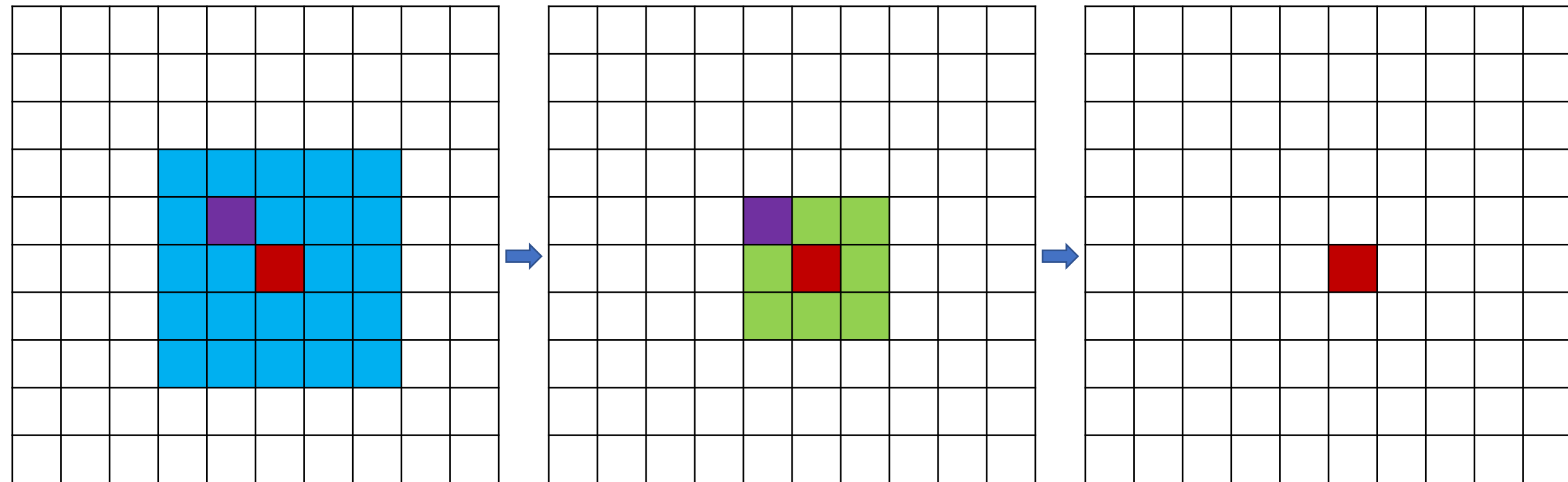
- Возьмём пиксель в итоговом изображении (после свёрточных слоёв)
- От какой части входного изображения зависит значение в этом пикселе?

Receptive field



Поле восприятия: 3 x 3

Receptive field



Поле восприятия: 5 x 5

Receptive field

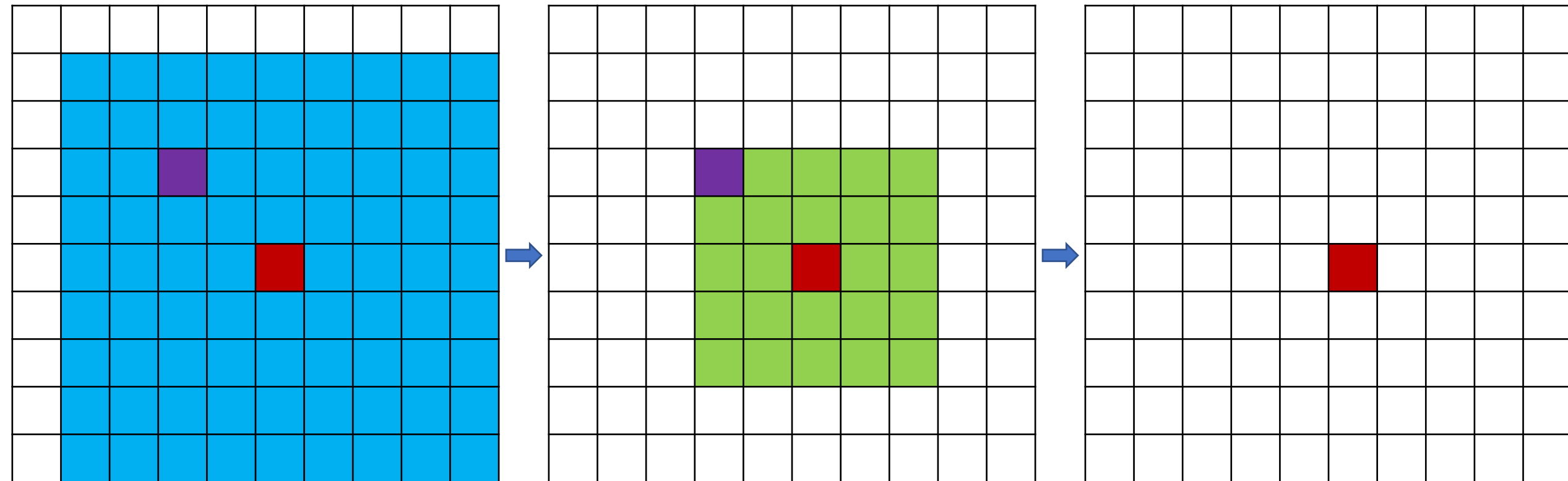
Поле восприятия для свёртки 3 x 3:

- После 1 свёрточного слоя: 3 x 3
- После 2 свёрточных слоёв: 5 x 5
- После 3 свёрточных слоёв: 7 x 7

Receptive field

Поле восприятия для свёртки 5 x 5:

Receptive field



Поле восприятия: 9 x 9

Receptive field

Поле восприятия для свёртки 5 x 5:

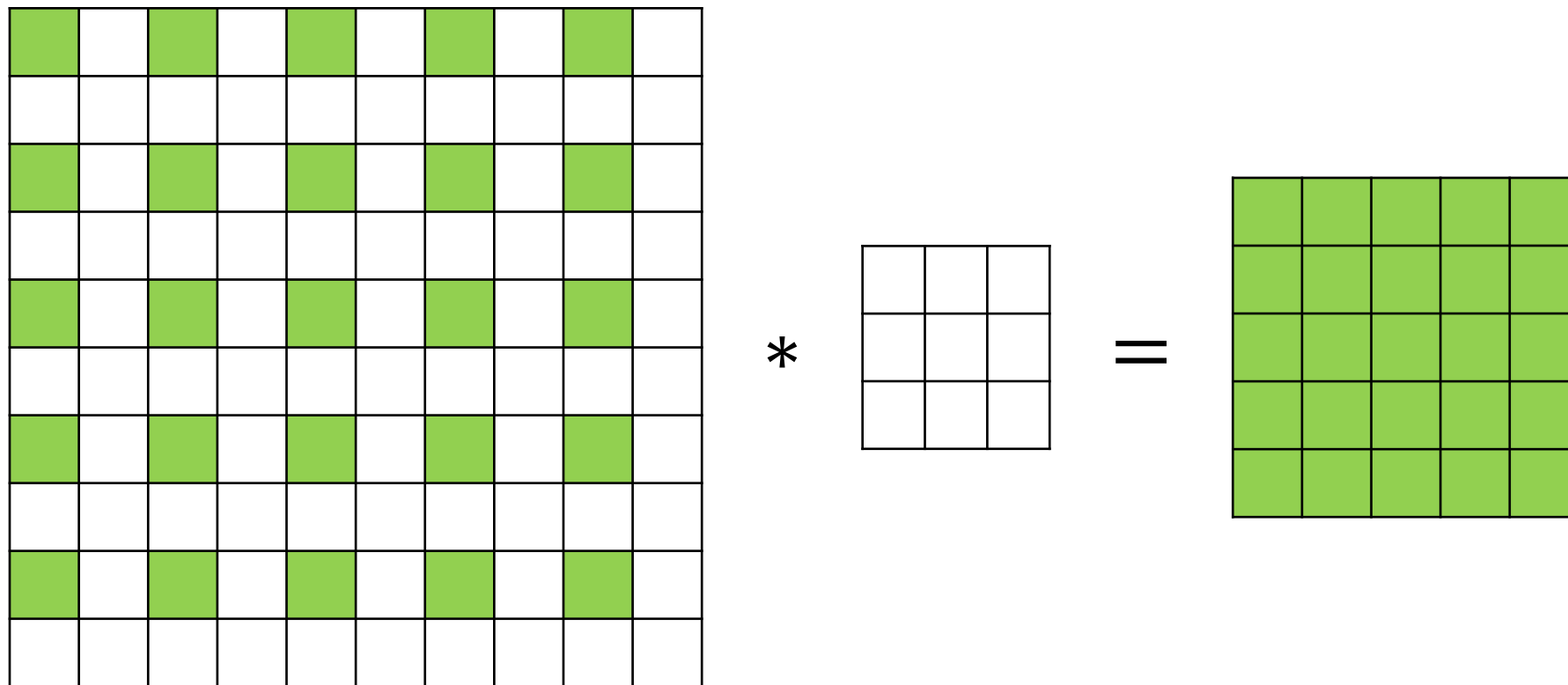
- После 1 свёрточного слоя: 5 x 5
- После 2 свёрточных слоёв: 9 x 9
- После 3 свёрточных слоёв: 13 x 13

Нужно очень много слоёв, если изображение размера 512 x 512

Почему фильтры маленькие

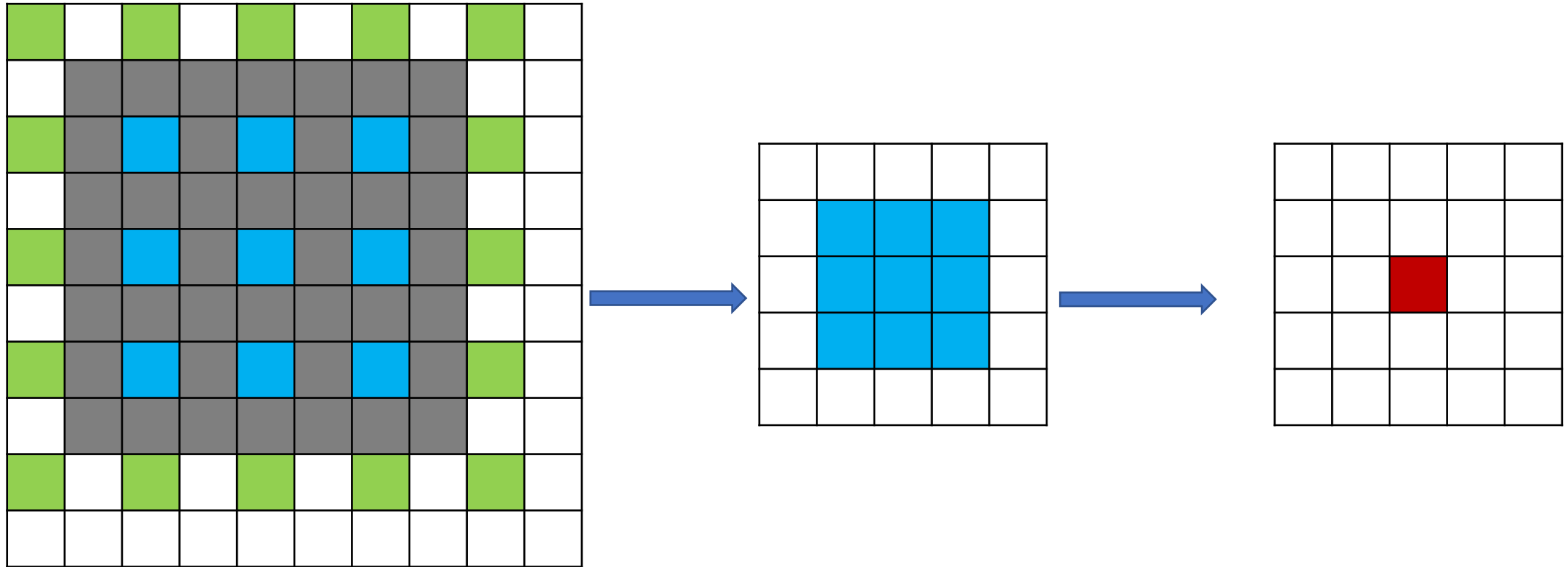
- Три слоя 3×3 дают поле восприятия 7×7
 - как один слой 7×7
- Параметров меньше:
 - один слой 7×7 : 49
 - три слоя 3×3 : $3 \cdot 9 = 27$ — почти вдвое меньше!
- Нелинейностей больше: три вместо одной → модель выразительнее
- Стандартом надолго стали свёртки 3×3 (VGG, 2014)

Свёртки с пропусками (strides)



$$s = 2$$

Свёртки с пропусками (strides)



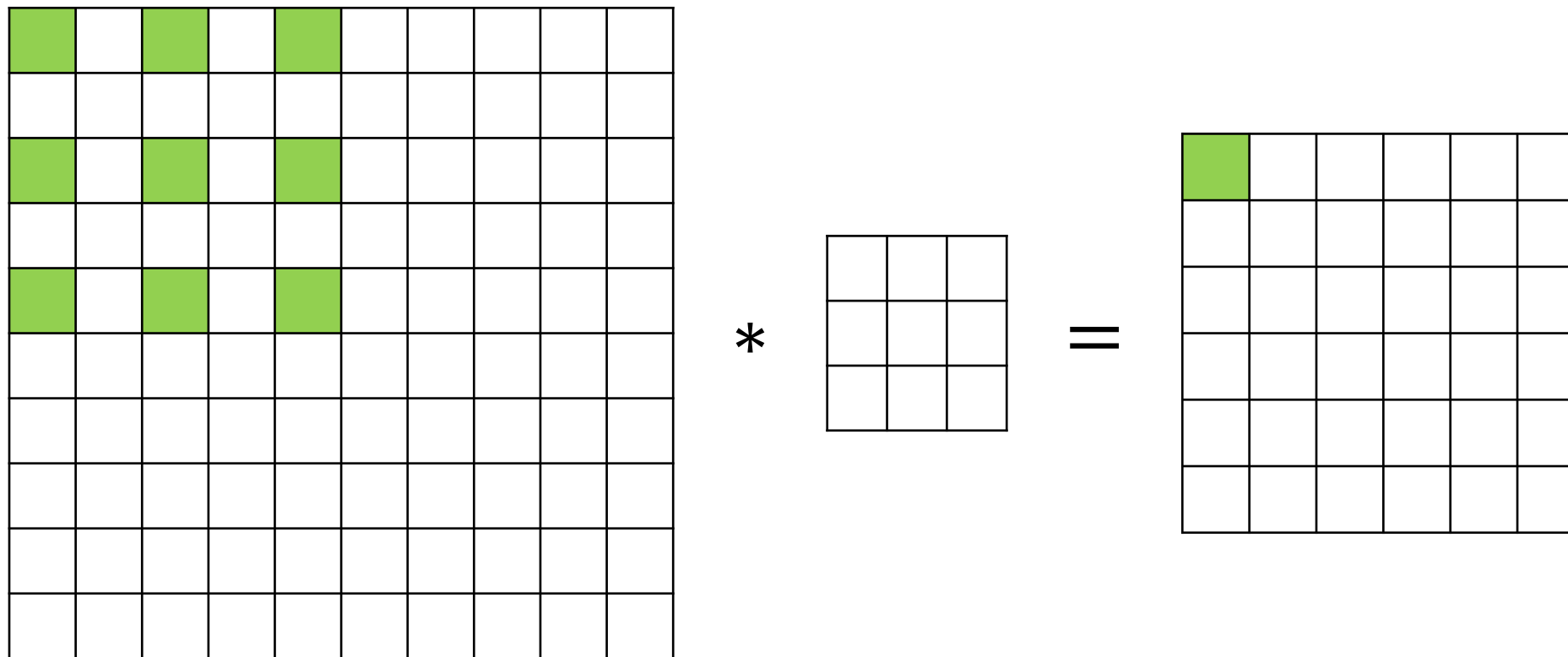
Поле восприятия: 7 x 7

Свёртки с пропусками (strides)

Подробности про подсчёт размера поля:

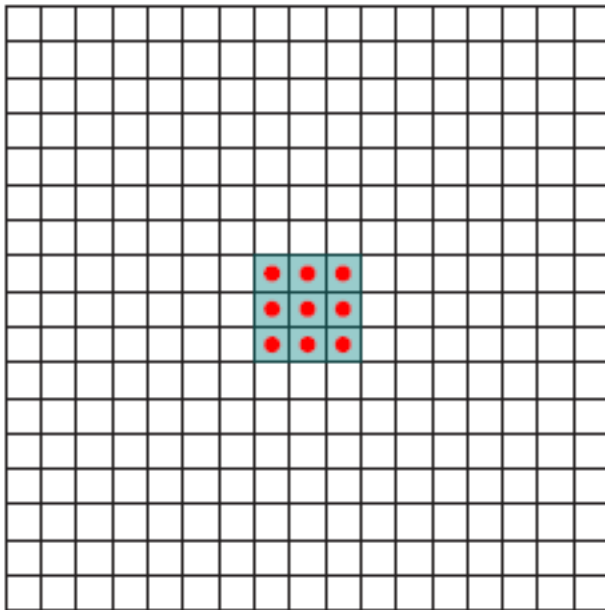
<https://distill.pub/2019/computing-receptive-fields/>

Dilated convolutions («раздутые» свёртки)

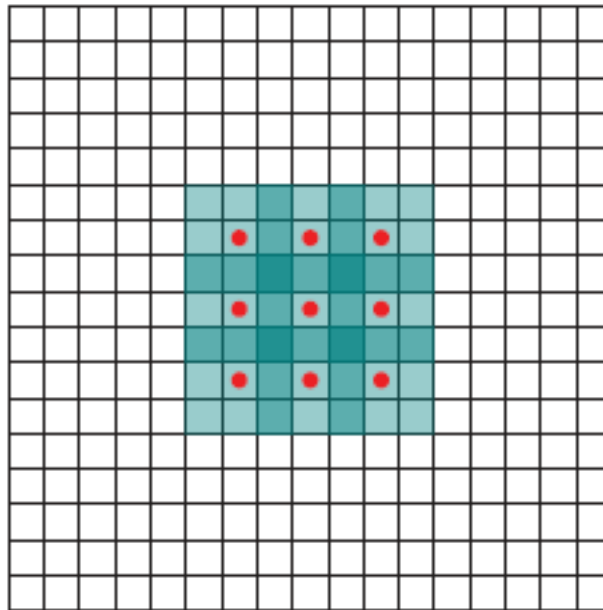


$$l = 2$$

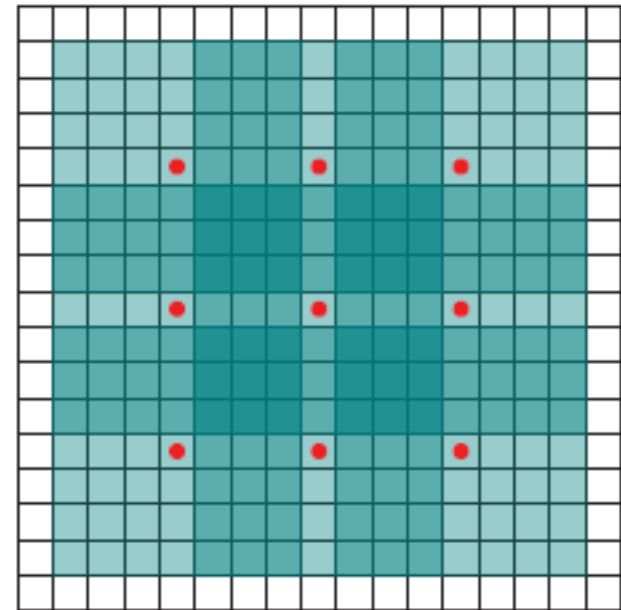
Dilated convolutions («раздутые» свёртки)



$l = 1$



$l = 2$



$l = 4$

Stride vs dilation

- Оба увеличивают поле восприятия, не добавляя параметров
- Stride: уменьшает изображение → дешевле, но теряем разрешение
 - Классификация, эмбединги
- Dilation: размер сохраняется → дороже, зато ответ в каждом пикселе
 - Сегментация, звук (WaveNet)

Pooling

1	0	2	1	0	0
0	1	3	2	1	2



1	3	2

Max-pooling с фильтром 2x2

Pooling

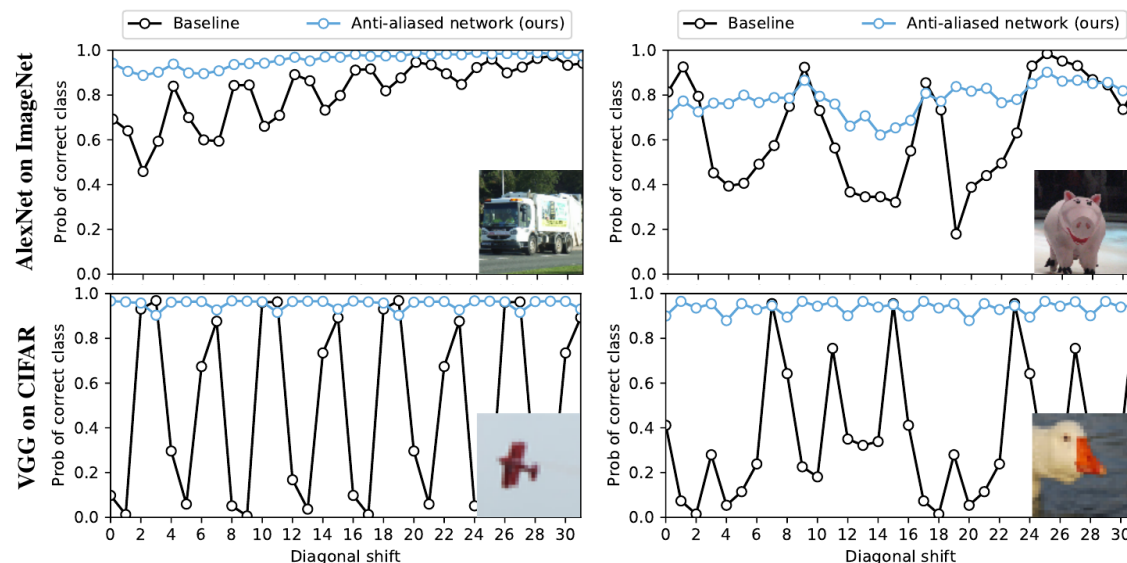
- Разбивает изображение на участки $n \times m$ и считает некоторую статистику в каждом участке (обычно максимум или среднее)
- Существенно сокращает размер изображения (значит, увеличивает поле восприятия следующих слоёв)
- Не имеет параметров
- Каждый канал обрабатывается отдельно

Pooling в PyTorch

- `nn.MaxPool2d(kernel_size)`, `nn.AvgPool2d(kernel_size)`
- Шаг по умолчанию равен размеру окна: `MaxPool2d(2)` уменьшает картинку вдвое
- `nn.AdaptiveAvgPool2d(1)` — одно число на канал при любом размере входа (global average pooling)

Инвариантность к сдвигам — не совсем

- stride и pooling прореживают карту
- из-за этого сдвиг картинки на 1 пиксель может заметно изменить ответ сети
- причина — алиасинг
- можно починить: размыть карту перед прореживанием (BlurPool)

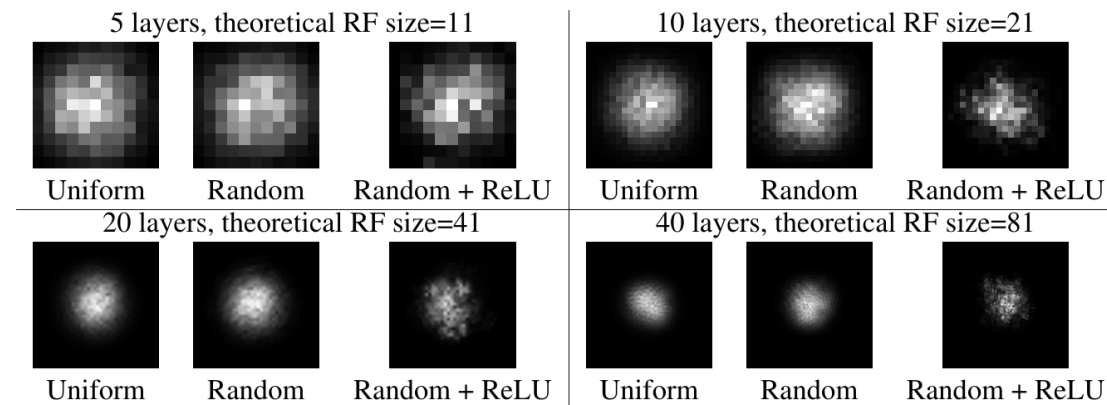


Зачем это всё?

- Важно следить за тем, чтобы последние свёрточные слои имели размер поля восприятия, сравнимый со всей картинкой

Effective receptive field

- Смотрим, как выход в центре зависит от пикселей входа (производная выхода по пикселю)
- Сильнее всего влияет на выход центр, к краям спадает
- Теоретическое поле растёт с числом слоёв n линейно, а эффективное — только как $\sqrt{n}$
- Вывод: поле восприятия должно быть даже больше, чем размер изображения
- После обучения эффективное поле восприятия становится шире



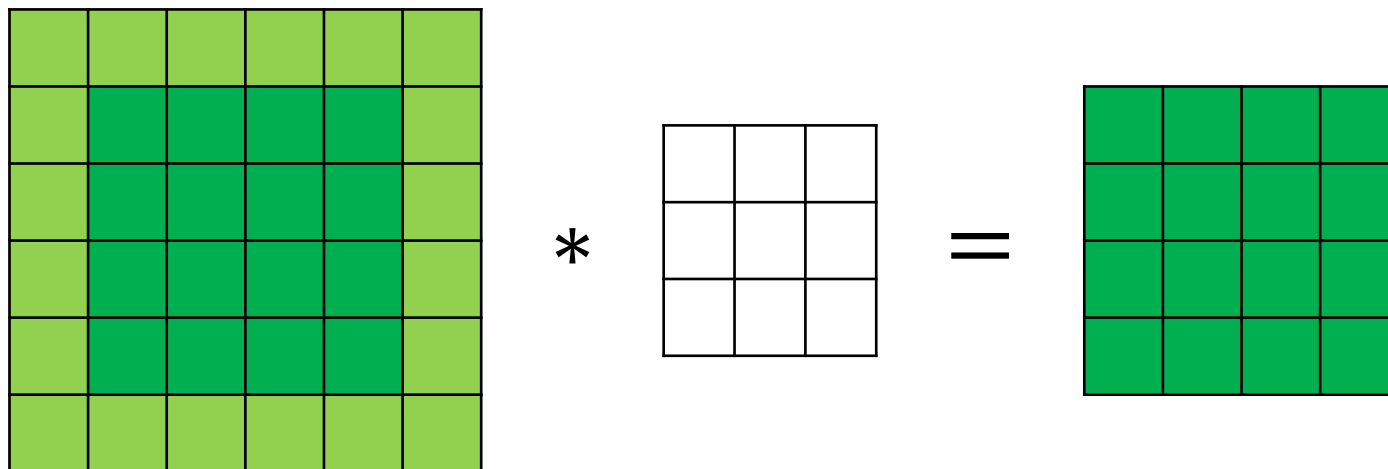
Свёртки 3×3; квадрат — теоретическое поле восприятия, яркость — влияние пикселя на выход

Padding

Свёртки

- Если применять свёртку по формуле, то выходное изображение будет меньше входного

Свёртки



Valid mode

- При честном подсчёте свёрток пиксели на краях не оказывают большого влияния на результат

Не увидим, что фильтр имеет хороший отклик при помещении центра в этот пиксель

0	1				
1	1				

*

0	0	1
0	0	1
1	1	1

Zero padding

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

*

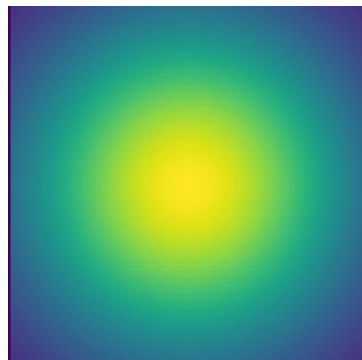
[illegible]

Zero padding

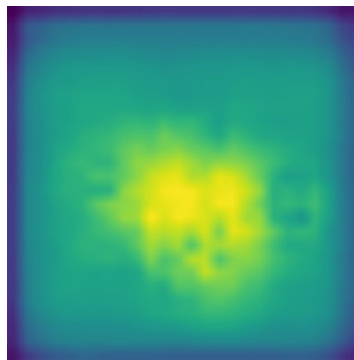
- Добавляем по границам нули так, чтобы посчитанная после этого свёртка в `valid mode` давала изображение такого же размера, как исходное
- Есть риск, что модель научится понимать, где на изображении края — можем потерять инвариантность
- И это происходит!

Zero padding выдаёт положение

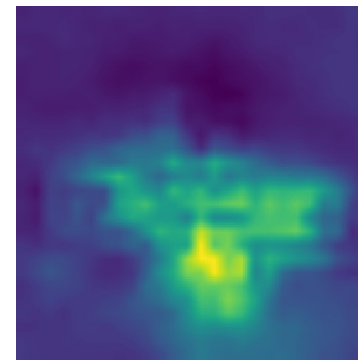
- Берём обученную VGG16, замораживаем
- Добавляем поверх одну свёртку 3×3
- Учим её рисовать одну и ту же карту для любой картинки: светлый центр, тёмные края
- Карта не зависит от содержимого → получится, только если признаки VGG знают координаты
- С паддингом получается (корреляция 0.81), без паддинга — заметно хуже (0.36)



Цель



VGG с паддингом



VGG без паддинга

Reflection padding

[illegible]

[illegible]

Reflection padding

- Не получится легко находить края изображения
- Но теперь модель может начать находить зеркальные отражения и подбирать фильтры под них

Replication padding

[illegible]

[illegible]

Replication padding

- Пиксель на границе равен ближайшему пикселю из изображения
- Модель всё ещё может настроиться под паттерны, которые возникают из-за такого паддинга

Circular padding

Строка 1 2 3 4 5 6, паддинг 2:

	слева	строка	справа
zeros	0 0	1 2 3 4 5 6	0 0
reflect	3 2	1 2 3 4 5 6	5 4
replicate	1 1	1 2 3 4 5 6	6 6
circular	5 6	1 2 3 4 5 6	1 2

- Circular: картинка склеена по краям, как тор
- Имеет смысл только для периодических данных: панорамы 360°, глобус и т.д.

Размер выхода

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

k — размер фильтра, p — паддинг, s — шаг

- $k = 3, p = 1, s = 1$: $32 \rightarrow 32$ (размер сохраняется; в общем случае $p = (k - 1)/2$)
- $k = 5, p = 0, s = 1$: $32 \rightarrow 28$ (valid mode)
- $k = 3, p = 1, s = 2$: $32 \rightarrow 16$
- $k = 7, p = 3, s = 2$: $224 \rightarrow 112$ (первый слой ResNet)

Резюме

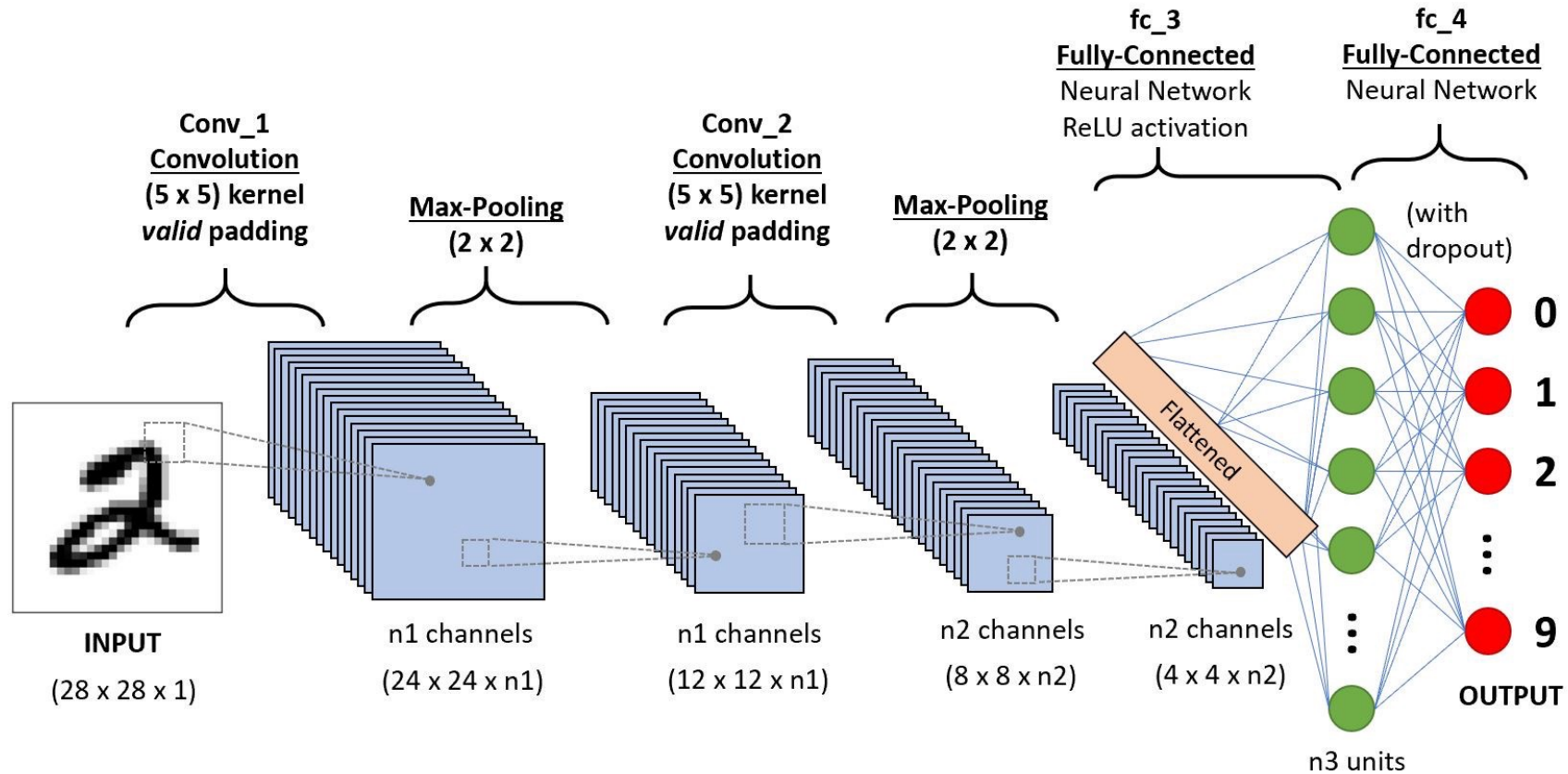
- Паддинг позволяет контролировать размер выходных изображений
- Паддинг позволяет учитывать даже объекты на краях
- Разные типы паддингов допускают разные способы переобучения под края

Структура свёрточных сетей

Свёрточный слой

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + \textcolor{red}{b}_t$$

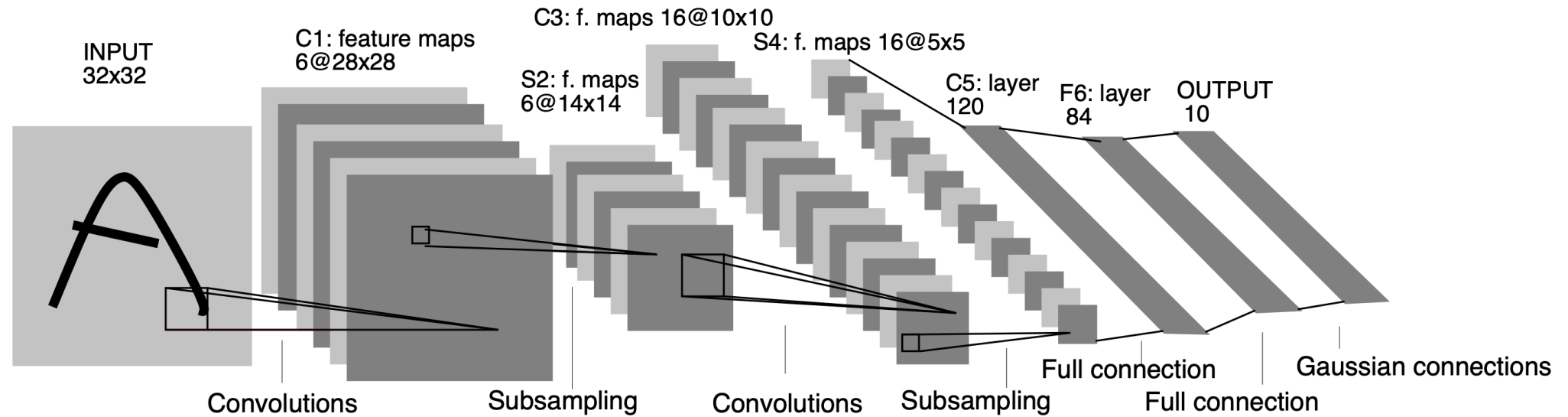
Типичная архитектура



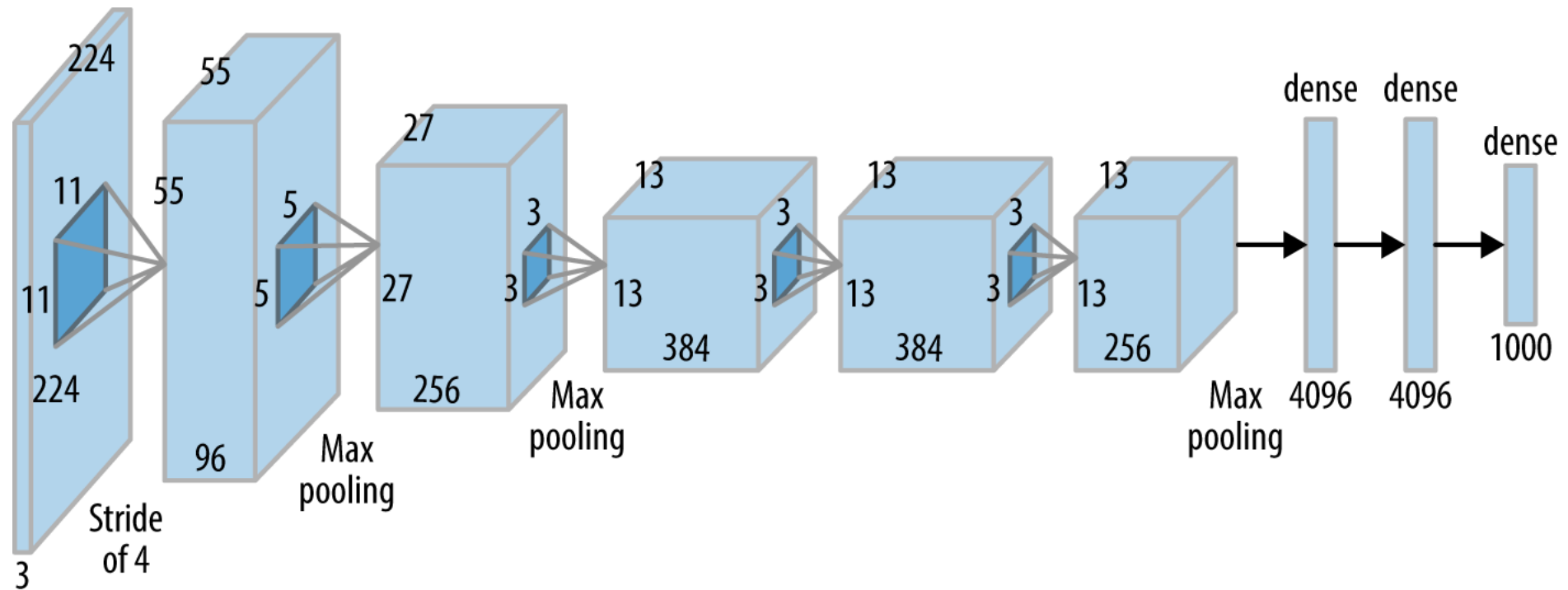
Типичная архитектура

- Последовательное применение комбинаций вида «свёрточный слой → нелинейность → pooling» или «свёрточный слой → нелинейность»
- Выпрямление (flattening) выхода очередного слоя
- Серия полносвязных слоёв
- Так устроены ранние сети (LeNet, AlexNet, VGG). В современных:
 - вместо flatten и серии FC-слоёв — global average pooling и один линейный слой
 - max-pooling остался только в начале сети, дальше свёртки со stride=2

LeNet



AlexNet



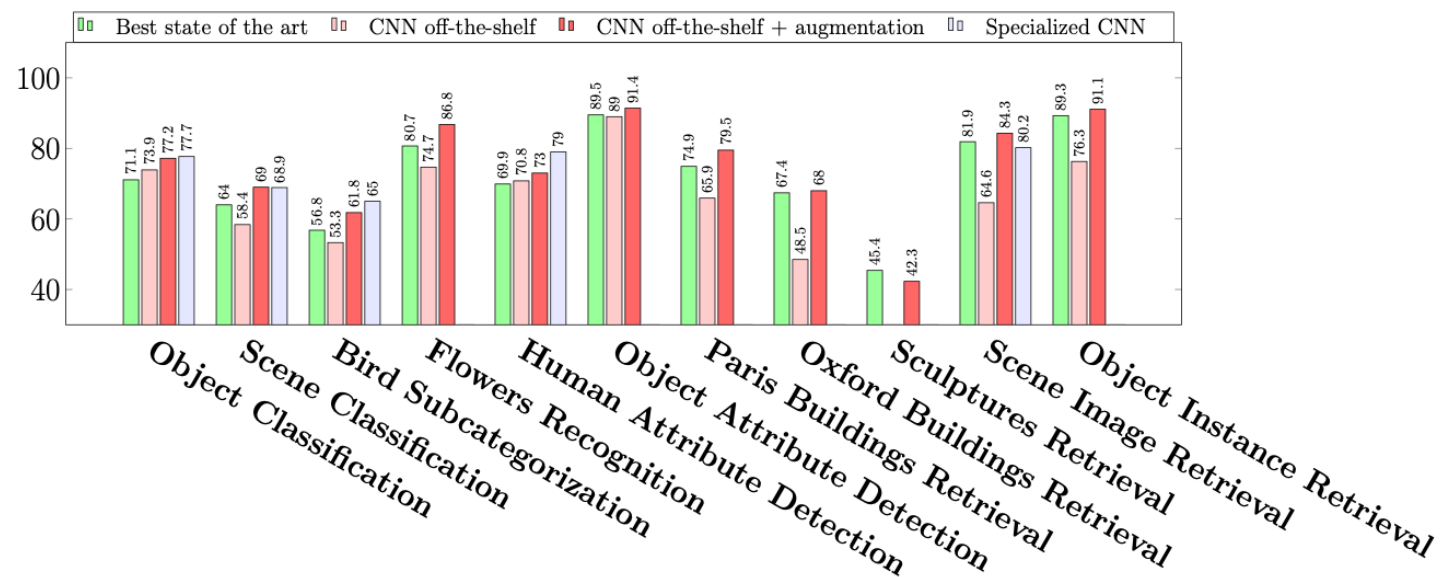
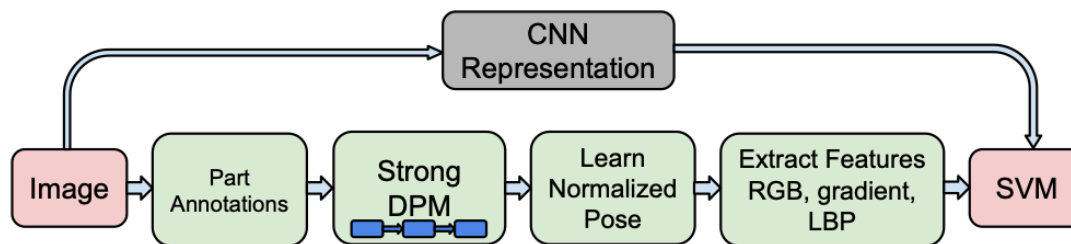
Параметры в AlexNet

- Всего ≈ 61 млн параметров
- Свёрточные слои $\approx 2,3$ млн
- Полносвязные слои $\approx 58,6$ млн (96%!)
 - первый из них: $6 \cdot 6 \cdot 256 = 9216$ входов $\times$ 4096 выходов $\approx 37,7$ млн
- Global average pooling: $256 \times 6 \times 6 \rightarrow 256$ чисел $\rightarrow$ линейный слой
 - ResNet-50: $2048 \rightarrow 1000$, ≈ 2 млн весов из 25,6 млн

Представления с последних слоёв

- Важное наблюдение: выходы последних слоёв являются хорошими признаковыми описаниями изображений
- Полезны во многих задачах
- Например, поиск похожих изображений

Представления с последних слоёв



Сейчас: кодировщики изображений

- Сеть учат ради хороших признаков на огромных данных
- CLIP: картинки вместе с подписями, 400 млн пар
 - задача: угадать, какая подпись к какой картинке относится (в батче из N пар найти N правильных среди N^2 сочетаний)
 - признаки картинки и текста попадают в одно пространство → поиск картинок по тексту
- DINOv2 / DINOv3: вообще без разметки
 - задача: по маленькому кусочку картинки предсказать то, что сеть-«учитель» выдаёт по всей картинке
 - DINOv3 — 1,7 млрд изображений, 7 млрд параметров